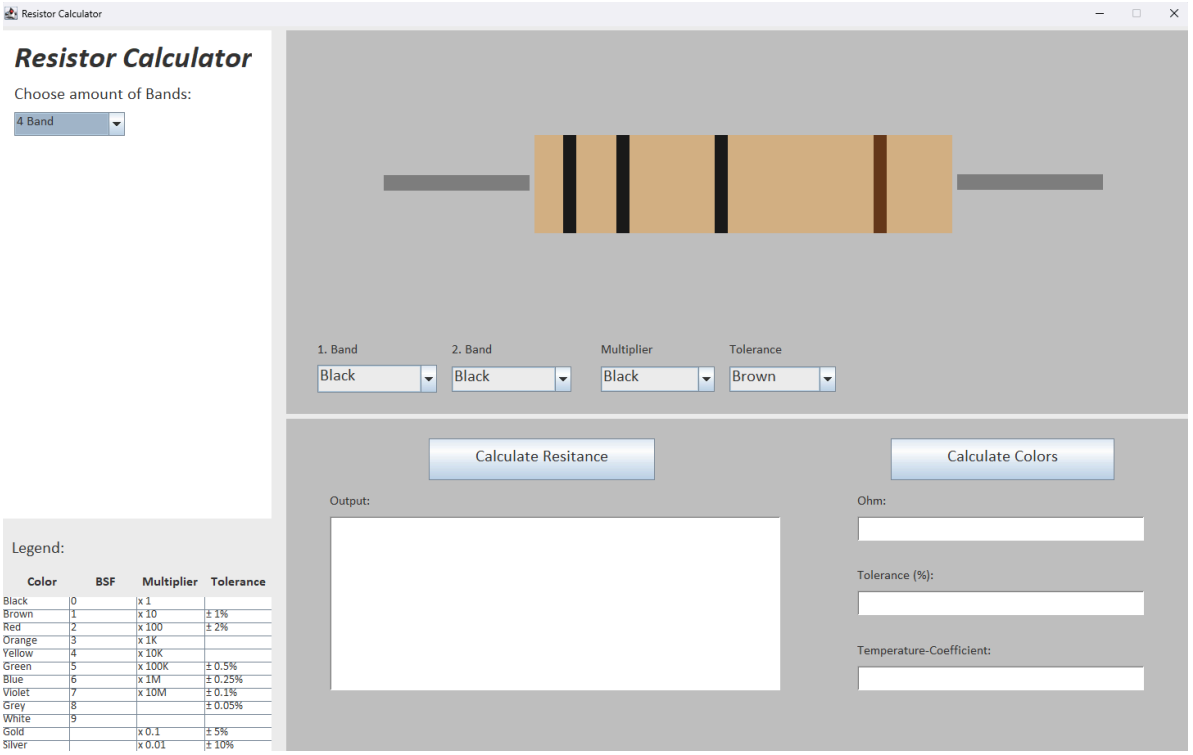
Umwelt-Campus Birkenfeld | HOCHSCHULE TRIER

Java-Bibliothek zur Darstellung und Umrechnung von Widerstands-Farbcodes

Eine Hausarbeit im Rahmen des Wahlpflichtmoduls Java SS2024

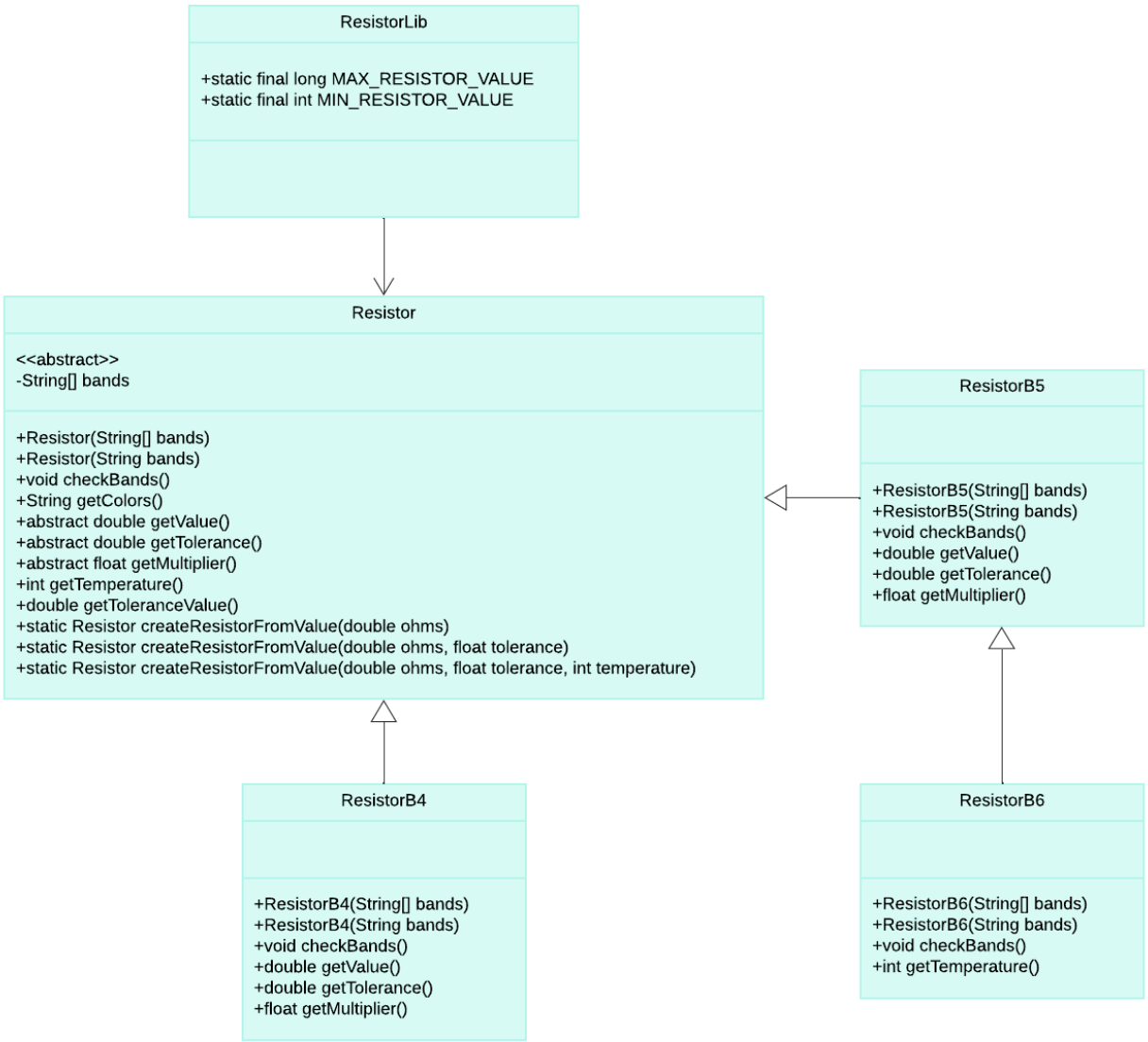


Florian Wittmann: flwt4651@umwelt-campus.de
Matthias Beck: s19d15@umwelt-campus.de
Philipp Göttel: s18de0@umwelt-campus.de

Design:

Grundstruktur

Unsere Resistor Lib ist wie folgt aufgebaut:



HashMaps oder Enums

Zur Codierung der Widerstandsbänder wurden HashMaps verwendet, da diese eine einfache und flexible Zuordnung zwischen Farbe und ihrem entsprechenden Wert bietet und vice versa. Zudem können die Hashmaps einfach erweitert werden, falls dies vom jeweiligen Anwender genutzt wird. somit können weitere Farben hinzugefügt oder aber bestehende Farben in anderen Sprachen implementiert werden. Zudem bieten HashMaps eine effiziente Möglichkeit, Schlüssel-Wert-Paare zu speichern und zu suchen. Gerade die Suchoperatoren ist äußerst performant. Des Weiteren sind HashMaps im Vergleich zu Enums besser geeignet Float und Double Werte zuzuordnen, da Enums hauptsächlich für diskrete Werte gedacht sind. Zuletzt wird durch die Verwendung von HashMaps die Logik zur Erstellung und Verarbeitung von Widerständen vereinfacht, da nicht für jede HashMap eine eigene Methode implementiert werden muss. Lediglich bei der Typsicherheit mussten wir etwas besser aufpassen.

Tests

Die Tests sind so aufgebaut, dass sie die Robustheit sowie die Funktionalität der **`ResistorLib-Klasse`** umfangreich überprüfen. Für jede Klasse, die Teil der **`ResistorLib`** ist, wurde ein umfangreiches Testszenario entworfen, das die Anzahl der entsprechenden Farbbänder sowie die Berechnungen der entsprechenden Widerstände auf Fehler, Toleranzwerte sowie Grenzwerte prüft.

Ebenso wurden grundlegende Tests zur validen Eingabe hinzugefügt. Zum Beispiel haben wir unlogische Eingaben wie zu kurze bzw. zu lange String-Eingaben für die einzelnen Konstruktoren abgefangen. Dies gewährleistet, dass die Klassen korrekt auf ungültige Eingaben reagieren und entsprechende Ausnahmen werfen.

Des Weiteren gibt es Tests, die sicherstellen, dass ungültige Farbangaben abgefangen werden und die entsprechende Position der falschen Eingabe in der Fehlermeldung angezeigt wird.

Ein Testen der String-Arrays sehen wir als obsolet an, da die eingegebenen Strings immer in ein String-Array umgewandelt werden.

Ein Problem gab es jedoch bei dem Test **`testResistorValueWithFloatOhms()`**. Aufgrund eines internen Rundungsfehlers können wir eine korrekte Ausgabe nicht zu 100 % garantieren. Selbst nach dem Einbau eines Lambda-Toleranzwertes von 0,0000001 kann es intern noch zu Problemen kommen.

GUI

1. Verwendung von Swing
   Swing ist ein etabliertes GUI-Toolkit für Java, das eine breite Palette von Komponenten bietet und plattformunabhängig ist. Es ermöglicht die Erstellung komplexer Benutzeroberflächen mit relativ geringem Aufwand und guter Performance.

2. Erstellung der GUI in einer eigenen Klasse, die von JFrame erbt
   Durch das Erben von *JFrame* wird die GUI-Klasse direkt als Fenster dargestellt. Dies erleichtert die Strukturierung und Verwaltung der GUI-Komponenten innerhalb eines einzigen Fensters.

3. Verwendung des EventQueue zur Initialisierung der GUI
   Der EventQueue.invokeLater-Aufruf stellt sicher, dass die GUI-Erstellung im Event-Dispatch-Thread (EDT) erfolgt. Dies ist wichtig, um Thread-Sicherheitsprobleme bei der GUI-Erstellung zu vermeiden.

4. Einsatz von GroupLayout zur Anordnung der Komponenten

GroupLayout bietet flexible Möglichkeiten zur Positionierung und Größenanpassung der GUI-Komponenten. Dies erleichtert die Erstellung dynamischer Layouts, die sich an verschiedene Fenstergrößen und -auflösungen anpassen können.

5. Verwendung von JComboBox-Elementen zur Auswahl von Farbringen, Multiplikator, Toleranz und Temperaturkoeffizient

JComboBox ermöglicht eine übersichtliche und kompakte Darstellung der Auswahlmöglichkeiten. Die Benutzer können schnell und intuitiv die gewünschten Optionen auswählen.

6. Dynamische Sichtbarkeit der JComboBox-Elemente basierend auf der Auswahl der Bandanzahl

Durch die Anpassung der sichtbaren Elemente je nach ausgewählter Bandanzahl wird die Benutzeroberfläche übersichtlich und benutzerfreundlich gestaltet. Nicht benötigte Optionen werden ausgeblendet, um Verwirrung zu vermeiden.

7. Verwendung von JPanel-Elementen zur strukturierten Anordnung und Gruppierung von Komponenten

JPanel bietet eine Möglichkeit, verwandte GUI-Komponenten zusammenzufassen und zu organisieren. Dies erleichtert die Verwaltung und das Layout der Benutzeroberfläche.

8. Einsatz von JLabel zur Beschriftung der Auswahlfelder

JLabel-Komponenten bieten eine einfache und klare Möglichkeit, Eingabefelder und Auswahlen zu beschriften, was die Benutzerführung verbessert.

9. Verwendung von JTextArea für Eingaben und Ausgaben

JTextArea ermöglicht die Anzeige und Eingabe von mehrzeiligem Text. In diesem Fall wird es zur Anzeige von Ausgabewerten und zur Eingabe von Widerstandswerten, Toleranzen und Temperaturkoeffizienten verwendet.

10. Einbindung von ActionListener zur Handhabung von Benutzeraktionen

ActionListener-Schnittstellen ermöglichen die Reaktion auf Benutzerinteraktionen, wie z.B. das Klicken von Schaltflächen oder die Auswahl von Dropdown-Optionen. Dies ist notwendig, um die Funktionalität der GUI zu implementieren.

11. Ästhetische Anpassungen durch Verwendung von Schriftarten und Farben

Die Anpassung der Schriftarten (Font) und Hintergrundfarben (Color) verbessert die visuelle Attraktivität und Benutzerfreundlichkeit der Anwendung. Eine konsistente und ansprechende Gestaltung trägt zur besseren Lesbarkeit und Bedienbarkeit bei.

12. Fazit

Die Design-Entscheidungen für das GUI zielen darauf ab, eine funktionale, benutzerfreundliche und optisch ansprechende GUI zur Berechnung von Widerstandswerten zu erstellen. Durch den Einsatz von Swing-Komponenten und Layout-Managern wird eine flexible und erweiterbare Struktur ermöglicht, die sowohl die Bedürfnisse der Benutzer als auch die technischen Anforderungen erfüllt.

Benutzeranleitung:

Die **`ResistorLib`**-Bibliothek bietet Werkzeuge zur Arbeit mit Widerständen. Sie ermöglicht das Erstellen und Manipulieren von Widerständen, basierend auf deren Farbcodes oder mithilfe des Resistenzwertes, optional auch der Toleranz und des Temperaturkoeffizienten.

Installation:

Um die **`ResistorLib`**-Bibliothek verwenden zu können, muss diese dem Projekt hinzugefügt werden. Dies erfolgt typischerweise durch das Hinzufügen der JAR-Datei zur Projektbibliothek oder durch die Integration in ein geeignetes Build-Tool (t.B. Gradle oder Maven).

Verwendung:

1. Erstellen von Widerständen

Es gibt unterschiedliche Möglichkeiten einen Widerstand zu erstellen. Zum einen durch Angabe des Farbcodes als zusammenhängenden String, wobei die Farben durch ein Komma getrennt werden, zum anderen durch Angabe eines String-Arrays, in dem jedes Element des Arrays die Farbbezeichnung des jeweiligen Bandes ist.

```
ResistorLib.Resistor resistorB4 = new ResistorLib.ResistorB4(new String[]{"red","violet","yellow","gold"});
ResistorLib.Resistor resistorB5 = new ResistorLib.ResistorB5(new String[]{"brown","black","red","gold","brown"});
ResistorLib.Resistor resistorB6 = new ResistorLib.ResistorB6(new String[]{"blue","gray","black","brown","gold","red"});
```

Zusätzlich lässt sich ein Resistor aber auch durch Angabe des Widerstandes in Ohm erzeugen. Optional kann auch die Toleranz (in %) und der Temperaturkoeffizient (in ppm/°C) angegeben werden.

```
ResistorLib.Resistor resistor = ResistorLib.Resistor.createResistorFromValue(470);
ResistorLib.Resistor resistorWithTolerance = ResistorLib.Resistor.createResistorFromValue(4700, 2f);
ResistorLib.Resistor resistorWithToleranceAndTemp = ResistorLib.Resistor.createResistorFromValue(4700, 2f, 100);
```

2. Abfrage von Widerstands Eigenschaften

Einmal erstellt, können die Eigenschaften des Widerstandes abgefragt werden.

```
System.out.println("Widerstandswert: " + resistor.getValue() + " Ohm");
System.out.println("Toleranz: ±" + resistor.getTolerance() + "%");
System.out.println("Multiplikator: " + resistor.getMultiplier());
System.out.println("Temperaturkoeffizient: " + resistor.getTemperature() + " ppm/°C");
```

3. Weitere Funktionen

Neben den Widerstands Eigenschaften kann auch der eigentliche Farbcode des Widerstandes ausgegeben und der Wert der Toleranz in Ohm ermittelt werden.

```
System.out.println("Farben der Bänder: " + resistor.getColors());
System.out.println("Toleranzwert: " + resistor.getToleranceValue() + " Ohm");
```

Fehlerbehandlung:

Die **`ResistorLib`**-Bibliothek überprüft automatisch die Gültigkeit der angegebenen Farben, Toleranzen und Temperaturkoeffizienten. Zudem prüft sie auch, ob ein Ohm Wert im angegebenen Wertebereich zwischen 0Ω und 9.99GΩ liegt. Bei ungültiger Eingabe wird eine ‘IllegalArgumentException’ mit Verweis auf das ungültige Band geworfen.

Testbeispiel:

- Wert: 470Ω
- Bandfarben: Gelb, Violett, Braun, Gold
- Multiplier: 10
- Toleranz: 5 %

Testablauf 1:
Ruft der Benutzer die Funktion createResistorFromValue(ohms, tolerance) auf, so erstellt diese Methode ein geeignetes Farbband für den Resistor. Der Farbcode Gelb, Violett, Braun, Gold wird anschließend dem Konstruktor des Resistors mitgegeben. Dieser erstellt den repräsentativen 4 Band Resistor.

Testablauf 2:
Erstellen eines Resistors über den entsprechenden Konstruktor. Ruft der Nutzer den ResistorB4 (string) auf und übergibt den String mit den oben genannten Farben, indem die einzelnen Farben mit Komma getrennt sein müssen.Danach wird das entsprechende Resistor Objekt mit den anhand der Farben errechneten Werten.

Testablauf 2:
Erstellen eines Resistors über den entsprechenden Konstruktor. Ruft der Nutzer den ResistorB4 (string[]) auf und übergibt den String mit den oben genannten Farben, indem die einzelnen Farben mit Komma getrennt sein müssen. Danach wird das entsprechende Resistor Objekt mit den anhand der Farben errechneten Werten.

Testablauf 4:
GUI: Kennt der Benutzer lediglich Farben, sowie die Anzahl der Bänder des Resistors. Kann er mithilfe der Combobox oben links die Anzahl der Bänder angeben. Hat er dies getan, füllt er anschließend die Comboboxen unter dem Resistor Bild aus. Sind bei allen Comboboxen die entsprechende Farbe ausgewählt, kann er nun auf den Button "Calculate Resistance" klicken, was zu einer Ausgabe mit den entsprechenden Werten in dem darunterliegenden TextArea führt.

Testablauf 5:
GUI: Kennt der Benutzer die entsprechende Werte eines Resistors, aber nicht die entsprechende Farbe der Bänder des Resistors kann er diese in die entsprechenden Inputfelder unten rechts eintragen. Kennt er nicht alle Werte, werden dementsprechende Default Werte genommen, die Anzahl Ohms ist jedoch zur Berechnung notwendig. Klickt er nach Eingabe der Werte auf den "Calculate Colors" Button wechselt der oben dargestellte Resistor seine Farben und die berechneten Werte erscheinen ebenso in dem links liegenden Textarea.

Eidesstattliche Erklärung:

Wir erklären hiermit, dass wir die vorliegende Projektarbeit selbst verfasst haben und dass wir dazu keine anderen als die angegebenen Hilfsmittel und Quellen verwendet haben. Alle Quellcode- und Textstellen, die wörtlich oder sinngemäß aus anderen Werken übernommen wurden, sind als solche gekennzeichnet. Mit Hilfe von künstlicher Intelligenz erstellter Text oder Quellcode ist als solcher mit Art und Umfang der Unterstützung dokumentiert. "Die Arbeit hat in gleicher oder ähnlicher Form noch keiner anderen Prüfungsstelle vorgelegen.“

Unterschriften der Projektteilnehmer:

Mathias Beck

Philipp Göttel

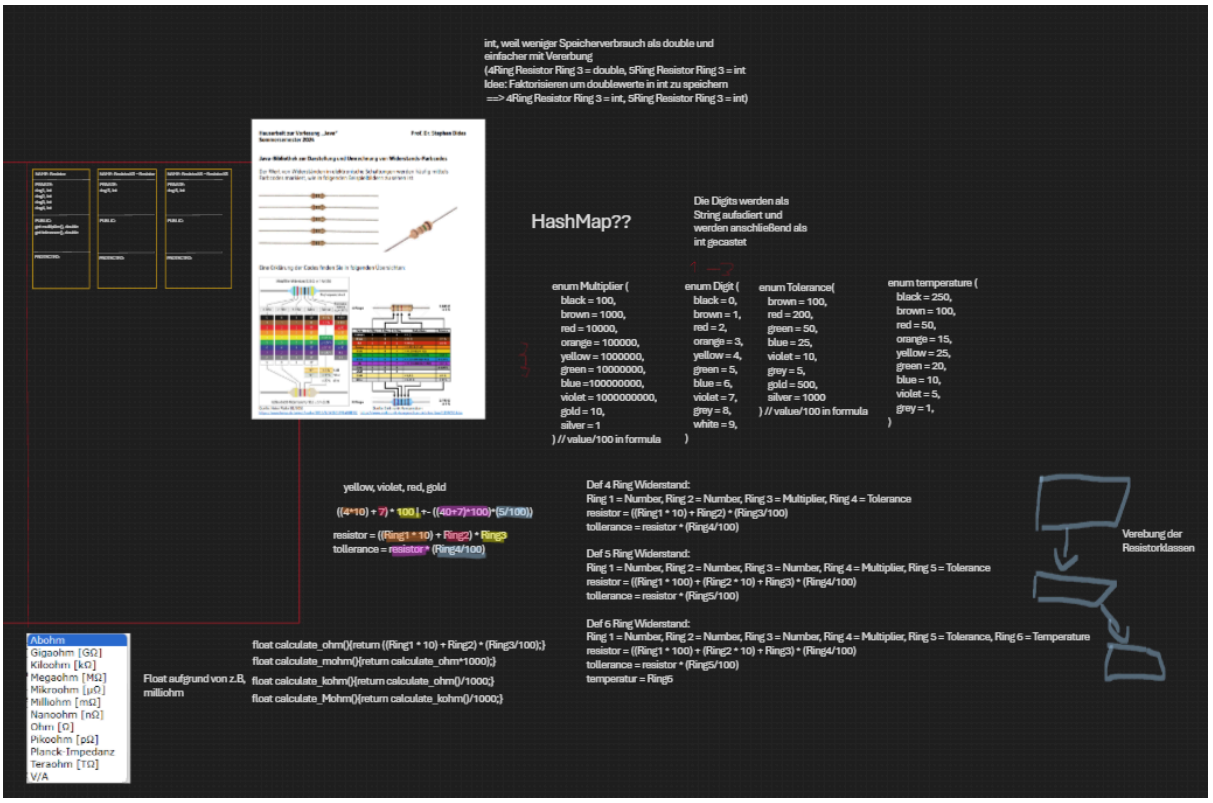
Florian Wittmann

Hilfsmittel:

Da zuvor keiner der Beteiligten mit JavaDocs gearbeitet hat, haben wir uns diesbezüglich zu Beginn von GPT 4.0 beraten lassen, wie genau der Aufbau auszusehen hat und haben diesen dann eigenständig fortgeführt. Die entsprechende Stelle wurde im Code gekennzeichnet (ResistorLib.java Zeile 309-317).

Zusätzlich haben wir eine Stelle im Code des GUI von GPT 4.0 generieren lassen. Diese ist auch als generiert gekennzeichnet (Gui.java Zeile 710-713).

Wie bereits erwähnt, gibt es noch ein Rundungsproblematik im Code, welche wir versucht haben mittels eines Lösungsansatzes von StackOverflow zu lösen. Wir haben diesen jedoch nicht übernommen da wir uns sicher sind, dass die Problematik nicht während der Rechnung von getToleranceValue entsteht sondern bei der übergabe des Ergebnisses an die Funktion AssertEquals(). Den Lösungsansatz haben wir jedoch im Code auskommentiert stehen lassen, um ein besseres Verständnis bezüglich unserer Fehlersuche zu zeigen. (ResistorLib.java Zeile 254-275, Stack Overflow. (n.d.). How to round a number to n decimal places in Java. In *Stack Overflow*. Abgerufen am 11. Juli 2024, von https://stackoverflow.com/questions/153724/how-to-round-a-number-to-n-decimal-places-in-java)

Appendix:

Planung der Hauptfunktion zum Umwandeln der Farbcodes in Dezimalwerte.